HNSW Algorithm: Efficiently Searching Vector Databases

8 min read · Apr 3, 2024



Ajay Mallya Follow

Listen Share



Image Generated by Bing Copilot

A brief preface...

As with all other posts of mine, I begin with a brief preface to illuminate my motivations for writing this. I'm a deeply curious undergraduate applied mathematics student who loves CS, LLMs, math, and any related theoretical underpinnings. This article discusses the motivations and build of the Navigable Hierarchical Small Worlds (HNSW) algorithm that enables lightning fast searches of vector databases. In writing this blog, I primarily referred to this leading and foundational HNSW paper in the field, but I'll try to refrain from using too much jargon. I hope this offers a sufficient treatment of the HNSW algorithm for anyone. The next few short sections will review some pertinent basics before we get to the intuitive breakdown of the HNSW algorithm, so bear with me. It will be worth the review, especially if you are more non-technical and seek a balanced blend of both detail and conceptual understanding.

> *Everything in this article will be simplified so it is more digestible to any curious minds, and later on I'll create some very deep technical blogs because that's when things get* really *interesting. Anyway, hopefully this provides a fruitful introduction to the algorithm.*

Vector Databases

The first type of database to be aware of is the **relational database**. Think of this as rows and columns storing data. For example, in a simple relational database, rows could represent different dogs, and each column for any given row could represent the characteristics of each dog (e.g. name, color, etc.). This set of rows and columns forms *one table* in a relational database, and there can be multiple tables.

Vector databases store unstructured data in vectors, enabling them to easily contain images, videos, etc. I'll write a post about how vector embeddings are created in the first place at a later time. Relational databases are useful for more precise data querying (e.g. business or transactional data), whereas vector databases are useful for accurate, fast querying when searching through unstructured data. Thus, vector databases are critical for AI applications that deal with unstructured data.

Nearest Neighbor Overview

At a very high level, nearest neighbor (NN) methods dictate that, given a search query for a particular data point, we look for the value stored in a vector database that is most similar to our query. This is how recommendation systems and search engines quickly find similar images to your search engine query. Let's say that I have a vector database full of celebrity images. If I search for Michael Jackson, various pictures of Michael Jackson will be returned to me because those are most similar to

my query. I have chosen to leave out more specific dependencies such as embedding models, etc. for simplicity's sake. One searching method is to inefficiently traverse the entire database for an exact match, comparing every data point to our query value until we find an exact match. Another more efficient method is to search for values that are more similar with a faster process (such as HNSW) and find the most similar value(s) (Approximate Nearest Neighbors, aka ANN). The latter does not require a screening of every value.

Graphs and Proximity Graphs

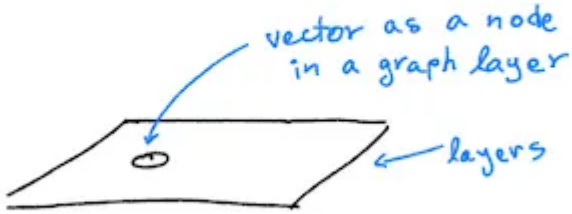
In computer science, graphs are a useful way to represent connections and relationships between datapoints. These are different from run-of-the-mill 2D and 3D graphs. For example, a graph could represent your family tree and portray how everyone is related. Graphs have vertices and edges. In this family tree example, the vertices could represent family members (e.g. mother, son) and the edges could represent their relationships to one another (e.g. "mother of," "son of").

A **proximity graph** connects vertices based on proximity (distance apart). As we'll see, the HNSW algorithm builds a graph and provides a neat way to traverse the graph to search for the most similar results for a given query.

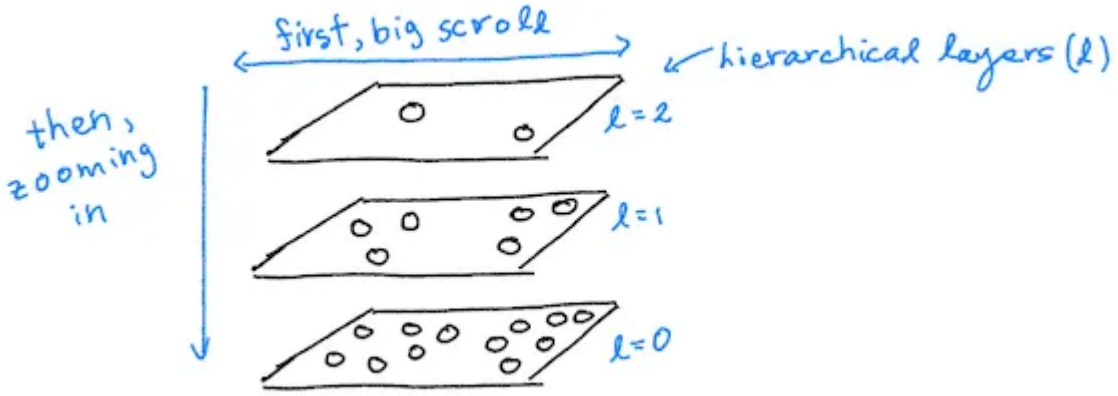
Hierarchical Navigable Small World: High Level

Before diving in, let me present an analogy that I've heard a couple of colleagues use. Let's say your location on Google Maps is zoomed in on Los Angeles, CA. To find New York, NY on the map, you can either stay zoomed in and scroll many times to get all the way to New York, *or* you can zoom out first, make a quick swipe across the country, and zoom in on New York. The former is inefficient as it requires more steps, whereas the latter is much faster. The latter is representative of HNSW, and I touched on this in the above "Nearest Neighbor Overview" section. With this intuition settled, let's dive right into the algorithm.

HNSW: Inserting Vertices/Vectors



We start with a database of vectors. Each vector is a vertex of the HNSW graph, illustrated with a circle (see above), and vertices exist on layers, illustrated with planes. There are different levels of layers in the constructed HNSW graph (hence the term, “hierarchical”) in which the vectors are inserted, with each layer illustrated as a plane (see below image).



Let’s first explore how the vector is inserted. Here are some parameters that are significant in the following explanation:

- **L** is the total number of layers we have. The top layer has value ℓ=L-1. So for 3 layers in a graph, the top layer is ℓ=2. The below layers will be ℓ=1 and ℓ=0.
- **mL** is a value that dictates the odds of a vector being inserted at higher levels. The greater the mL value, the greater odds of a vector being inserted in higher levels. Conversely, the closer mL is to 0, the greater the odds of the vector being inserted at layer 0 (i.e. ℓ=0).

So if a vector gets ℓ=2 as its randomly chosen layer of insertion, it is added to this layer and the below two layers as well. Similarly, if a vector insertion gets ℓ=1, it is

Open in app ↗ Sign up Sign in

Medium Search

Get Ajay Mallya's stories in your inbox

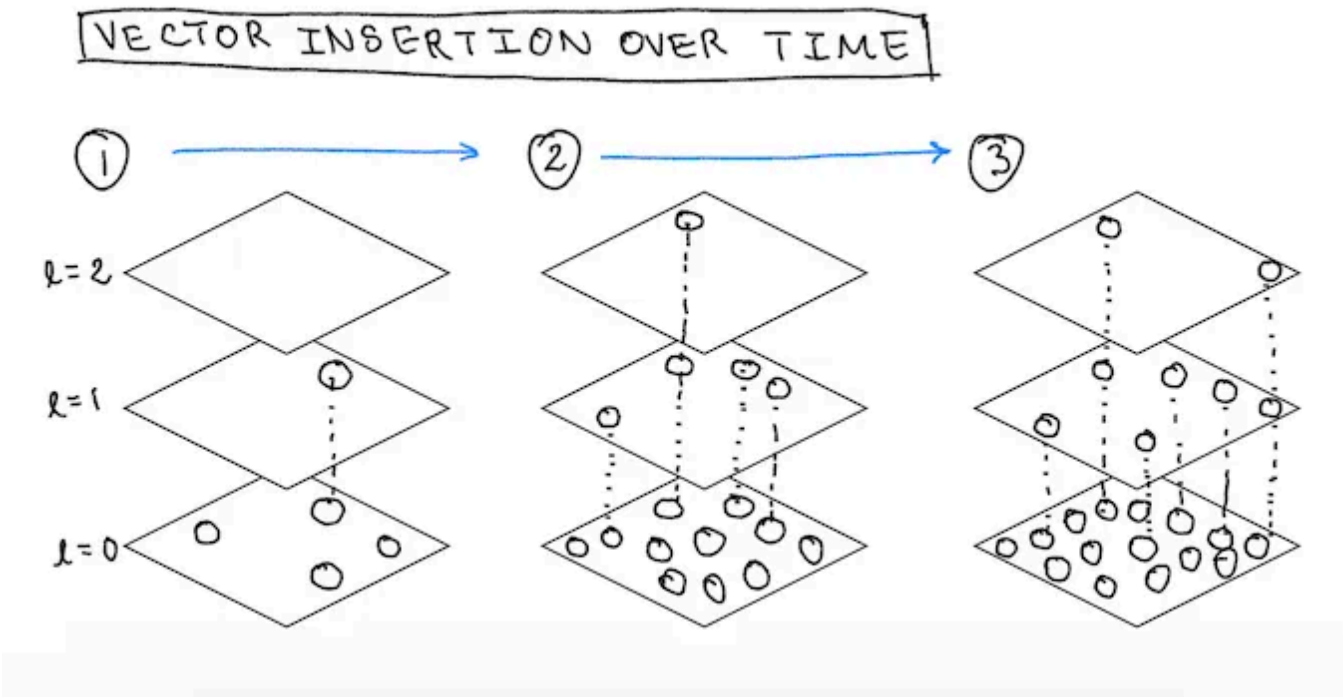
Join Medium for free to get updates from this writer.

Enter your email

Subscribe

Remember me for faster sign in

To make sure our graph has more vectors in the bottom layers than in the top layers, according to our mL definition above, we can adjust mL to be closer to 0. Eventually, we add all of our vectors to the graph, and over time we notice that there are more vectors in the bottom layers of the graph than in the top layers (see below). In the below visual, you will notice that the insertion of lower level vertices occurs much more often than the higher level vertices, resulting in significantly more lower level vertices by the end of the entire insertion process.



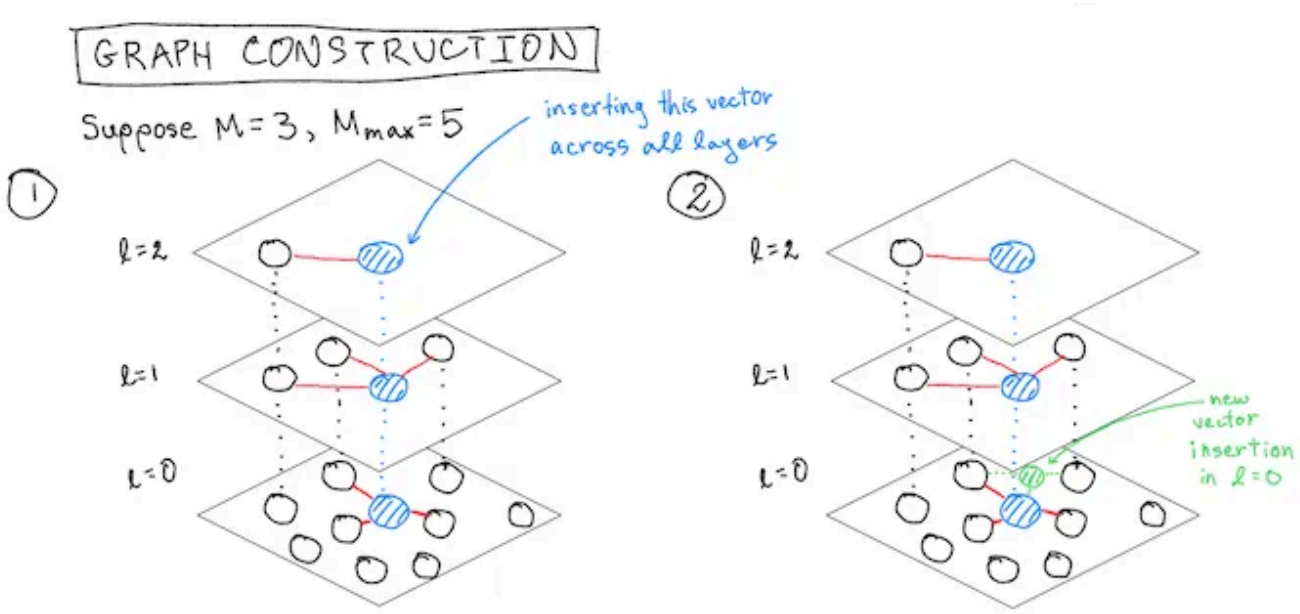
An important insight that assisted my initial understanding of HNSW is the following: *all* vectors exist in the bottom layer. A smaller percentage of all vectors exist in the next layer up, and an even smaller percentage exist in the highest layer.

> *Remark: See the original paper, linked at the onset of this post, to investigate how the optimal mL value of 1/ln(M), where M represents the number of NNs to return, influences the random selection of the maximum insertion layer for a given vector insertion. This involves some interesting probability concepts. I may add a related section here later on, but for now I exclude this for simplicity's sake.*

HNSW: Graph Construction

Now we get to adding connections (edges) between all of our inserted vectors (vertices) in our proximity graph. Here are two more relevant parameters for creating edges between nodes:

- **M** dictates the number of connections a vertex can have with nearest neighbors. In figure 1 of the below visual, the connections satisfy the preset M=3 because in any of the layers, our inserted vertex (in blue) only ever maxes out at 3 *outgoing connections* to other vertices. You see that in the top layer, it only has one outgoing connection, and that's totally fine because it's less than our maximum.
- **M_{max}** dictates the number of *maximum connections that a vertex can have in a given layer* when subsequent vectors are inserted throughout the insertion process. In figure 2 of the below visual, the connections satisfy the preset M_{max}=5 because when we add a new vertex (in green) to the bottom layer, our previous blue vertex only connects to the green vertex once, for a total of 4 connections for our blue vertex. Since our M_{max}=5, we can add more vectors in the bottom layer and if they are close enough, there can still be one more allowable connection to our blue vertex.



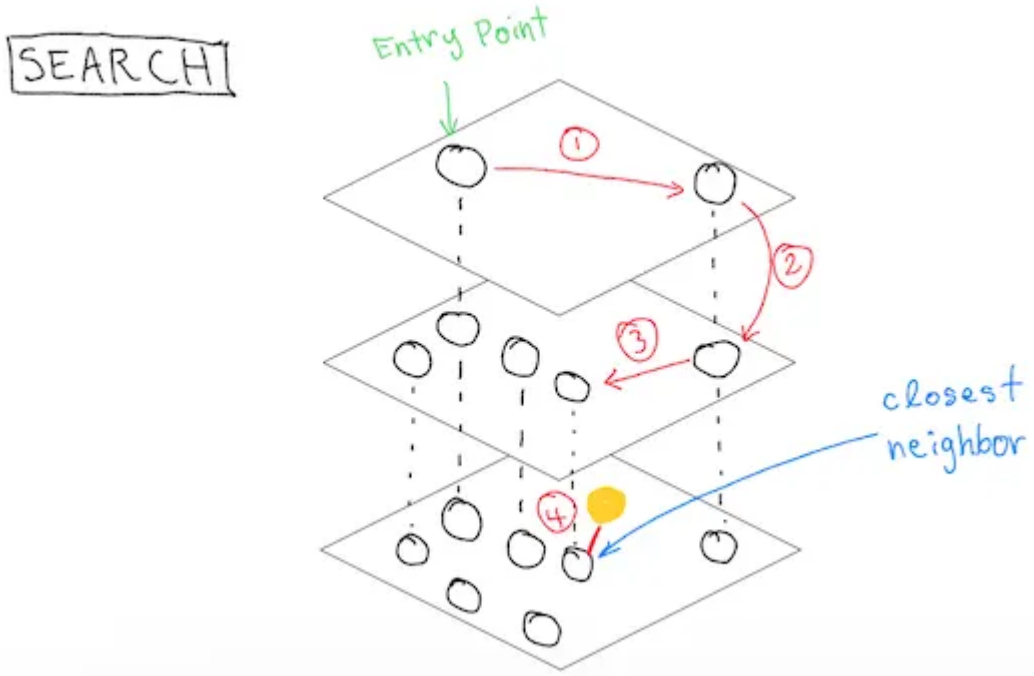
Through the above steps, we add connections (i.e. edges) to all of our vectors.

HNSW: Search

Say we have a vector that we're looking for. In the below visualization I drew it as a gold circle. The HNSW algorithm's objective is to find a value (i.e. a vertex) as close to the target as possible. Here is a list of steps that follows the trajectory illustrated below:

1. We begin at a pre-defined entry point in the top layer. In this top layer, we can see which of the vertices is closest to our target and move to that one. This is the first step of the "zooming in" in action. Instead of searching in the wrong section of our data, we save time and immediately move to the more relevant section. Very elegant step here.
2. Since this is the nearest neighbor in the top layer, we now drop to the next layer to find potential vertices that are closer to our target than our current vertex.
3. Indeed, we find a vertex that is closer, so this is now the nearest neighbor to our target.
4. Finally, we drop down to the base layer and see that this vertex is the nearest neighbor in the bottom layer as well.

Thus, we have found our target vertex/vector!



Concluding Thoughts

This is one of my favorite graph traversal algorithms thus far, and I'll certainly add more detailed mathematics later on, depending on the questions I receive and other interactions with this post. I hope this was a sufficiently intuitive, high level explanation of the Hierarchical Navigable Small Worlds algorithm (what a mouthful). More importantly, hopefully this offers some insight into how high-performance vector databases dealing with AI-related vector embedding data manage to efficiently pull queries in mere milliseconds from collections of potentially millions or billions of vectors.

Large Language Models | Vector | Vector Database | AI | Algorithms



Follow

Written by Ajay Mallya

111 followers · 5 following

lifelong learner, constant thinker b.a. applied mathematics @columbia graduate certificate program in AI @stanford